

CSCE 632 ACCESSIBLE COMPUTING · MODULE 11 · LECTURE NOTES

Games and Real-Time Systems

Where this track's work meets this course's requirements.

MODULE OUTCOMES

- Explain why games are a distinct accessibility problem.
- State the input, difficulty, and presentation options.
- State the photosensitivity and vestibular requirements.
- Explain accessibility in a rendering and engine context.
- Specify accessibility options for a real-time system.

1 Why games differ

A GAME'S DIFFICULTY IS DELIBERATE, WHICH MAKES SEPARATING INTENDED FROM ACCIDENTAL DIFFICULTY THE WHOLE PROBLEM

An application that is hard to operate is simply defective — a game that is hard to play may be working exactly as designed — which is a distinction no other interface in this course has to reason about and is why this module exists separately.

So the question for every barrier is whether the difficulty is the intended challenge or an incidental obstacle — a precise timing puzzle may be entirely the point, and a menu that requires a timed input is never the point and is a plain defect.

Which gives a usable rule: everything outside the intended challenge should be as easy as possible — menus, navigation, text, inventory management, saving, and settings — and it is remarkable how much of a typical game's difficulty lives there accidentally.

And inside the challenge, offer options rather than removing it — which preserves the design for the players who want it and makes the game finishable for those who cannot meet one particular demand, which is the settled position in the field and resolves the 'but difficulty is the art' objection.

2 The options

INPUT

- full remapping, including modifier keys
- no required simultaneous inputs
- toggle instead of hold, everywhere
- no required rapid repeated presses
- adjustable sensitivity and dead zones

DIFFICULTY

- separable: combat, timing, puzzle, resource
- not one slider governing all of them
- and adjustable mid-game, without penalty

PRESENTATION

- text size, independently of overall UI scale
- subtitles with speaker names (Module 04)
- high-contrast and colour-blind modes
- audio cues duplicated visually, and visual cues duplicated audibly

CAMERA AND MOTION

- field of view, shake, motion blur, head bob
- all adjustable or disableable (section 3)

Toggle instead of hold, everywhere — the cheapest single option on this list and one of the most **widely needed**, since sustained pressure is exactly what many motor conditions cannot provide (Module 05 §1). **Separable difficulty is the option most often requested and least often provided**: a player who wants the full combat challenge and cannot solve a timing puzzle is served by one slider per axis and not at all by a single easy-normal-hard choice.

AND THE COST IS ALMOST ENTIRELY IN RETROFITTING, WHICH IS AN ARCHITECTURE ARGUMENT

Remappable input is cheap if the input layer is abstracted behind named actions and expensive if raw key codes are scattered through the gameplay code — which is a decision made in the first week of a project and is very hard to reverse later.

Separable difficulty is cheap if the difficulty parameters are data and expensive if they are constants compiled into the behaviour — likewise a week-one decision, and likewise one nobody frames as an accessibility decision at the time.

And text scaling is cheap if the UI layout is responsive and very nearly impossible if it is pixel-positioned against a fixed design resolution — which is the same argument a third time.

So the accessibility cost of a real-time system is set by its architecture rather than by its accessibility work, which is Module 12 §3's retrofit argument in the form this track will actually meet it — and is the version to make to an engine programmer.

3 Photosensitivity and motion

FLASHING CAN INDUCE A SEIZURE AND LARGE-FIELD MOTION CAN INDUCE SICKNESS, AND BOTH ARE DESIGN-CONTROLLABLE

The general flash threshold limits how much of the screen may flash, how rapidly, and at what contrast — roughly, no more than three flashes per second over a large central area, with saturated red flashes treated more strictly, because red is specifically implicated.

And it is testable — analysis tools exist for exactly this purpose and are used in broadcast compliance — which makes it one of the very few requirements in this module with a numeric check rather than a judgement (Module 07 §4's testability point).

Vestibular symptoms come from large-field visual motion the body did not cause — camera shake, head bob, motion blur, forced camera movement, rapid field-of-view changes, and a narrow field of view — and the response is to make each of them individually adjustable, because which one triggers symptoms varies.

Plus respect the operating system's reduced-motion preference — which the platform already exposes and almost nothing reads — and which is a one-line check with a real effect, in web interfaces as much as in games (and is the single easiest item in this module).

4 Engines and rendering

- The UI layer needs an accessibility tree (Module 08) — and a game engine's custom-rendered UI usually does not have one at all, since it draws textured quads rather than declaring semantic elements — which is the largest single gap in this area.
- Which means menus, settings screens, and in-game text are frequently unreachable by a screen reader even where the gameplay itself is playable — and the menu is exactly where the accessibility options live, which is a particular irony and a real barrier.
- Contrast and colour requirements apply to the HUD (Module 03 §3) — and a HUD drawn over arbitrary rendered content has no fixed background, so it needs its own backing plate or outline rather than a contrast ratio computed against one screenshot.
- And the render settings are accessibility settings — motion blur, depth of field, bloom, chromatic aberration, and film grain all reduce legibility, sometimes severely — so each needs to be separately disableable rather than bundled into a quality preset.
- Which is CSCE 647's rendering features meeting this course directly: physically correct is not the same as legible — the same observation CSCE 679 Module 12 §4 makes about scientific volume rendering, arriving from the other side.

AND THE CLOSING POSITION

Games are where the accessibility argument is hardest to make and where the results are most visible — because a finishable game is an unambiguous outcome, unlike 'conformant', and players report it directly.

And the field has improved faster here than anywhere else in this course, driven by players and developers rather than by regulation — which is Module 12 §4's point about what actually causes change, and a genuinely encouraging counterexample to that module's general pessimism.

So the practical requirement for this track is five architectural decisions: abstract the input behind named actions, parameterise the difficulty as data, make the UI layout scale, expose the render settings individually, and respect reduced motion.

All five of which are cheap in week one and expensive in year two — which is the only accessibility argument that reliably lands with an engineering lead, and is therefore the one to make first (Module 12 §3).

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
The Game Accessibility Guidelines (free)	§2 — organised by implementation effort, which makes it usable mid-project rather than only at the start.
The AbleGamers APX player experience resources (free)	§1's intended-versus-incidental distinction , developed into design patterns.
Understanding WCAG 2.3.1 and 2.3.3 on flashes and motion (free)	§3's thresholds , with the derivation and the red-flash exception.
The Xbox and PlayStation accessibility guidelines (free)	§§2 and 4 as platform requirements — and these are what a publisher will actually be held to.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Take one game you know** and classify five difficulties as intended or incidental.
- 2 **Find a menu that requires a timed input.**
- 3 **Audit one of your own projects against §2's option list.**
- 4 **Replace every hold with a toggle option** in something you built.

- 5 **Separate one difficulty slider** into its component axes.
- 6 **Check whether your input layer is abstracted**, and estimate the remapping cost.
- 7 **Run a flash analysis** on any sequence you have built.
- 8 **Read the reduced-motion preference** and act on it, in one project.
- 9 **Try to reach your own menus with a screen reader**.
- 10 **Disable each render effect individually** and compare HUD legibility.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why do games need separate treatment?
- 2 State the intended-versus-incidental rule and its consequence.
- 3 What should happen inside the intended challenge?
- 4 Give five input options and five presentation options.
- 5 Which single option is cheapest and most needed?
- 6 Why is separable difficulty requested and rarely provided?
- 7 State the three architecture arguments.
- 8 Give the flash threshold and the red exception.
- 9 Name five sources of vestibular symptoms.
- 10 Why is a game engine's UI usually inaccessible, and why is that ironic?